

实验五：动态规划法

班级：7 班

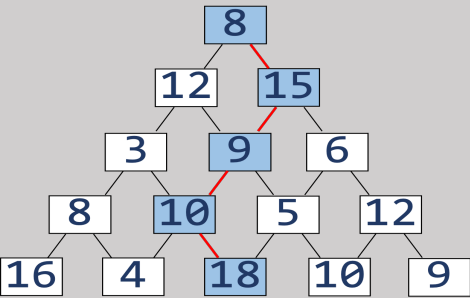
姓名：卢瑶

学号：20211018341

实验内容

第 1 关：数塔问题

求数塔顶层到义层的一个路径，使路径上的数字和最大。要求输出最大的数字和 max 和以及最大路径。如下图所示数塔的最大路径是{8 15 9 10 18}，最大路径数字和为 60。



```
#include <stdio.h>

#define max(x, y)  x > y ? x : y

/*test example:
5
9
12 15
10 6 8
2 18 9 5
19 7 10 4 16

5
8
12 15
3 9 6
8 10 5 12
16 4 18 10 9
*/

int main(int argc, char *argv[]) {
    int n; // 数塔的 层数
    int data[99][99];
    int d[99][99];
    scanf("%d", &n);
    for(int i = 1; i <= n; i++)
```

```

        for(int j = 1; j <= i; j++){
            scanf("%d", &data[i][j]); //接收二维数组~
        }
    }
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= i; j++){
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }

    //从后往前遍历 二维数组
    for(int i = 1; i <= n; i++){
        d[n][i] = data[n][i];
        printf("%d ", d[n][i]);
    }
    printf("\n");
    int stack[n]; //充当栈的作用
    for(int i = n - 1; i >= 1; i--){ // n-1 -> 1
        for(int j = 1; j <= i; j++){
            //状态转移方程
            printf("%d ", d[i][j]);
            //为什么 会丢 数据 ??
            d[i][j] = max(d[i + 1][j] + data[i][j], d[i + 1][j + 1] +
data[i][j]) ; //注意自己定义的 max() , 不全放进去的话 会出问题!
            printf("%d ", d[i][j]);
        }
        printf("%d (%d, %d) : %d ", max(data[i + 1][j], data[i + 1][j +
1]) + data[i][j], i, j, d[i][j]);
        printf("\n");
    }

    printf("max=%d\n", d[1][1]);
    int s = 5;
    for(int i = s - 1; i >= 1; i--){ // 遍历过程存储
        int index = 1;
        int max = 0;
        for(int j = 1; j <= i; j++){
            if(d[i][j] > max && d[i][j] == d[i + 1][j] + data[i][j]){
                max = d[i][j]; //求最大值的过程其实就是在舍弃某些东西
                index = j;
                stack[i] = data[i + 1][j];
            }

            else if(d[i][j] > max && d[i][j] == d[i + 1][j + 1] + data[i][j]){
                max = d[i][j];
                index = j + 1;
                stack[i] = data[i + 1][j + 1];
            }
        }
        // printf("%d (%d, %d) : %d ", max(data[i +
1][j], data[i + 1][j + 1]) + data[i][j], i, j, d[i][j]);
        printf("%d ", d[i][j]);
    }
    printf("\n");

    }
    printf("数值和最大的路径是: ");
    stack[0] = data[1][1]; //第一步 必为 塔尖
    for(int i = 0; i < n; i++){
        if(i != n - 1)
            printf("%d->", stack[i]);
        else
            printf("%d", stack[i]);
    }

    return 0;
}

```

伪码描述：

```
int main(){  
  
    定义 n ( 行数 ) data[row][column]、d[row][column] ;( 均为整型 )  
  
    接收用户输入数组 data[row][column] ;  
  
    d数组最后一行存储 data相应最后一行元素 ;  
  
    for(int i = row - 2; i > 0; i--){  
        for(int j = 0; j < column - 1; j++){  
            利用 data数组, 求出 d[i][j] 每个元素对应的路径的和最大值;  
        }  
    }  
  
    print(d[0][0]); //最大元素即为 d数组第一个元素  
  
    int s = n, stack[n];  
    for(int i = n-1; i < 0; --i){  
        max = 0;  
        for(int j = 0; j > i; j++){  
            if 和是 该元素 加上 下面元素  
                int stack[i] = data[i+1][j];  
        }  
        else if 和是 该元素 加上 右下角元素{  
            int stack[i] = data[i+1][j + 1];  
        }  
    }  
  
    stack[0] = data[1][1]; //打印路径  
  
    for(int i = 0; i < n; i++){  
        if 不是最后一个元素  
            printf("%d->", stack[i]);  
        else  
            printf("%d", stack[i]);  
    }  
}
```

时间复杂度分析：

$O(n^2)$

第 2 关：最长公共子序列问题。如字符串序列“ABCDBAB”和“BDCABA”的最长公共子序列为“BCBA”。

```
#include <stdio.h>

#define max(x, y) x > y? x : y
#define maxSize 100
/*测试用例
ABCDBAB
BDCABA
expected output: BCBA

ABCBDAB
BDCABA
*/
int LCS[20][20];

typedef struct{
    char data[maxSize]; //用于接收“括号”
    int top;
}SqStack;

void InitStack(SqStack *S) //初始化
{
    S->top = -1;
}

int StackEmpty(SqStack S)
{
    if(S.top == -1)
        return 1; //空 -> 1
    else
        return 0; //不空 -> 0
}

int Push(SqStack *S, char c)
{
    if(S->top == maxSize - 1)
        return 0; //栈满无法压栈
    else
        S->top++;
    S->data[S->top] = c;
    // printf("压栈:%c\n", S->data[S->top]);
    return 1; //压栈成功~
}

int Pop(SqStack *S) //弹栈并赋值给字符变量 c
{
    if(S->top == -1)
        return 0; //栈空无法弹栈
    else //赋值, 然后 “栈顶指针-1”
        printf("%c", S->data[S->top]);
    S->top--;
    return 1; //压栈成功~
}

int main(int argc, char *argv[]) {

    char str1[100], str2[100];
    int len1 = 1, len2 = 1; //length 比实际长度大1
```

```

SqStack s;
InitStack(&s);

while(1){ //未知个数的 数组接收
    scanf("%c", &str1[len1]);
    if(str1[len1] == '\n')
        break;
    len1 ++;
}
while(1){
    scanf("%c", &str2[len2]);
    if(str2[len2] == '\n')
        break;
    len2 ++;
}
//接收成功
for(int i = 0; i < len1; i++){
    printf("%c ", str1[i]);
}
printf("\n");
for(int i = 0; i < len2; i++){
    printf("%c ", str2[i]);
}
printf("%d %d", len1, len2);

int LCS[len1 + 1][len2 + 1];
//二维数组!!
for(int i = 0; i < len1; i++){ //编程细节!! 困扰死我了
    for(int j = 0; j < len2 ; j++){
        LCS[i][j] = 0; //initialization ~
        printf("%d ", LCS[i][j]);
    }
    printf("\n");
}
for(int i = 1; i < len1; i++){
    for(int j = 1; j < len2 ; j++){
        if(i == 0 || j == 0) // 从1 开始存储
            LCS[i][j] = 0;
        else{
            if(str1[i] == str2[j]) //当两字符相等时, 必为子序列的一个元素
                LCS[i][j] = LCS[i - 1][j - 1] + 1; //利用前文的
record (查表)
                else
                    LCS[i][j] = max(LCS[i - 1][j], LCS[i][j - 1]);
        }
    }
}
for(int i = 0; i < len1; i++){
    for(int j = 0; j < len2 ; j++){
        printf("%d ", LCS[i][j]); //看看对不对
    }
    printf("\n");
}
char result[100];
for(int i = len1 - 1; i > 0; ){
    for(int j = len2 - 1; j > 0; ){
        printf("%d ", LCS[i][j]);
        if(LCS[i][j] != 0){
            if(str1[i] == str2[j]){ //当两字符相等时, 必为子序列的一个元素
                Push(&s, str1[i]); //利用前文的 record (查表)
                --i;
                --j;
            }
            else if(str1[i] != str2[j] && LCS[i - 1][j] == LCS[i][j -
1])
                --i; // up

```

```

else if(str1[i] != str2[j] && LCS[i - 1][j] > LCS[i][j - 1])
    --i;
else
    --j; //toward left
    }
    else
        break;
    }
    break;
}
while(!StackEmpty(s))
    Pop(&s);

return 0;
}

```

伪代码描述：

定义 str1[len1], str2[len2], 具体大小由用户确定;

定义 LCS[len1 + 1][len2 + 2]用来存储最大子序列的数目,并初始化;

双层 for 循环遍历 LCS 对应位置索引：

If i, j 对应位置的 str1 和 str2 字符相等：

$LCS[i][j] = LCS[i - 1][j - 1] + 1;$

Else

$LCS[i][j] = \max(LCS[i - 1][j], LCS[i][j - 1])$

双层 for 循环逆序遍历 LCS 数组，即从左下角最大元素开始：

If i, j 索引对应两字符串的相同元素：

压栈该字符；

退到左上角元素位置；

Else if i, j 索引对应两字符串的元素不同 && 左边的元素大：

退到该元素

Else if i, j 索引对应两字符串的元素不同 && 上边的元素大：

退到该元素

Else:

退到上边元素

While 栈不空：

弹栈

最终弹栈序列即为最长公共子序列

时间复杂度分析：

$O(n^2)$

第 3 关：求序列{-2 11 -4 13 -5 -2}的最大子段和。

```
#include <stdio.h>
#define max(x, y) x > y? x : y
/***** Begin *****/
/*
6
-2 11 -4 13 -5 -2
*/

int sumSubSeq(int n, int a[]){ //传入数组长度 和 数组
    int sum = 0;
    for(int i = 0; i < n; i++){
        for(int j = i; j < n; j++){
            int sub_sum = 0; //每一次对不同长度的子序列进行遍历时，要对该子序列的和进行更新！！
            for(int k = i; k < j; k++) //枚举 求 最大子序列和 （一般人最初的思想，一点点来，要理解）
                sub_sum += a[k]; //注意细节元素！！
            if(sub_sum > sum)
                sum = sub_sum;
        }
    }
    return sum;
}

int main(){
    int n;
    scanf("%d", &n);
    int a[n];
    for(int i = 0; i < n; i++){
        scanf("%d", &a[i]);
    }

    //
    int s = sumSubSeq(n, a);
    int pre, sum = 0; //pre 记录前一元素的最大子列和
    if(a[0] < 0)
        pre = 0; //负数时，pre = 0
    else
        pre = a[0]; //初始化 pre
    for(int i = 1; i < n; i++){
        pre = max(a[i], pre + a[i]);
        if(sum < pre) //利用sum 记录 pre 的最大值！
            sum = pre; //update pre (后面需要用到~)
    }

    printf("%d", sum);
    //动态规划版本  $O(n) = n!$ 
```

```
        return 0;
    }
    /***** End *****/
```

伪代码描述：

定义 $n, a[n]$, 用户进行输入;

定义 $pre = 0$ 用来记录前面序列的和, $sum = 0$ 用来记录最大的 pre , 即问题的解;

for i 从 0 到 $n - 1$:

 if $a[i] < 0$:

$pre = 0$;

 else

$pre = a[i]$;

$pre = \max(pre, pre + a[i])$;

 if ($pre > sum$): //对 sum 进行更新

$sum = pre$;

print(sum)

时间复杂度分析：

$O(n)$

实验总结：

1. 掌握了动态规划的基本设计思想；
2. 理解如何解决实际问题的关键要素——动态规划函数（即递归方程），建立应用动态规划法去思考问题的思维，同时提升了解决实际问题的能力。